# LAPORAN PRAKTIKUM
# PEMROGRAMAN MOBILE
# MODUL 5


## Connect to the Internet

**Oleh:**

**Muhammad Guntur Ricky Adhitya NIM. 2410817310003**

**PROGRAM STUDI TEKNOLOGI INFORMASI**

**FAKULTAS TEKNIK**

**UNIVERSITAS LAMBUNG MANGKURAT**

**JUNI 2026**

# LEMBAR PENGESAHAN

# LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE

# MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

| | |
|---|---|
| Nama Praktikan | : Muhammad Guntur Ricky Adhitya |
| NIM | : 2410817310003 |

| | |
|---|---|
| Menyetujui,<br>Asisten Praktikum | Menyetujui,<br>Dosen Penanggung Jawab Praktikum |
| Muhammad Erza Raihan<br>NIM. 2310817210027 | Irham Maulani Abdul Gani, S.Kom., M.Kom.<br>NIP. 199710312025061009 |

# DAFTAR ISI

# DAFTAR GAMBAR

# DAFTAR TABEL

# SOAL 1

**Soal Praktikum:**

1. Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:
    a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
    b. Gunakan KotlinX Serialization sebagai library JSON.
    c. Gunakan library seperti Coil atau Glide untuk image loading.
    d. API yang digunakan pada modul ini adalah The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: https://developer.themoviedb.org/docs/getting-started
    e. Implementasikan konsep data persistence (aplikasi menyimpan data walau pengguna keluar dari aplikasi) dengan SharedPreferences untuk menyimpan data ringan (seperti pengaturan aplikasi) dan Room untuk data relasional.
    f. Gunakan caching strategy pada Room. Dibebaskan untuk memilih caching strategy yang sesuai, dan sertakan penjelasan kenapa menggunakan caching strategy tersebut.
    g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

**A. Source Code**

**1. Main**

*Tabel 1. Source Code MainActivity.kt Modul 5*

```
1  package com.mobil.modul5compose
2
3  import android.os.Bundle
4  import androidx.activity.compose.setContent
5  import androidx.activity.enableEdgeToEdge
6  import androidx.appcompat.app.AppCompatActivity
7  import androidx.navigation.compose.rememberNavController
8  import com.mobil.modul5compose.navigation.AppNavigation
9  import com.mobil.modul5compose.ui.theme.Modul3ComposeTheme
10
11 class MainActivity : AppCompatActivity() {
12
13     override fun onCreate(savedInstanceState: Bundle?) {
14         super.onCreate(savedInstanceState)
15         enableEdgeToEdge()
16
17         val app = application as Modul5Application
18
19         setContent {
20             Modul3ComposeTheme {
21                 val navController = rememberNavController()
22                 AppNavigation(
23                     navController = navController,
24                                         settingsRepository  =
   app.settingsRepository,
```

```
                    movieRepository = app.movieRepository
                )
            }
        }
    }
}
```

*Tabel 2. Source Code Modul5Compose.kt Modul 5*

```
package com.mobil.modul5compose

import android.app.Application
import androidx.room.Room
import com.mobil.modul5compose.data.SettingsRepository
import com.mobil.modul5compose.data.local.AppDatabase
import com.mobil.modul5compose.network.MovieRepository
import com.mobil.modul5compose.network.TmdbHttpClientFactory
import timber.log.Timber

class Modul5Application : Application() {
    lateinit var settingsRepository: SettingsRepository
    lateinit var movieRepository: MovieRepository

    override fun onCreate() {
        super.onCreate()

        if (BuildConfig.DEBUG) {
```

```
19              Timber.plant(Timber.DebugTree())
20          }
21
22          settingsRepository = SettingsRepository(this)
23
24                  val database = Room.databaseBuilder(this,
    AppDatabase::class.java, "movie_db")
25              .fallbackToDestructiveMigration(false)
26              .build()
27
28                                      val    httpClient    =
    TmdbHttpClientFactory(BuildConfig.READ_ACCESS_TOKEN).create
    ()
29               movieRepository = MovieRepository(httpClient,
    database.movieDao(), settingsRepository)
30      }
31  }
```

**2. Data**

*Tabel 3. Source Code MovieResponse.kt Modul 5*

```
1   package com.mobil.modul5compose.data
2
3   import kotlinx.serialization.SerialName
4   import kotlinx.serialization.Serializable
5
6   @Serializable
7   data class MovieResponse(
8       val id: Int,
```

```
9     val title: String? = null,
10    val overview: String? = null,
11      @SerialName("poster_path") val posterPath: String? =
    null,
12    @SerialName("release_date") val releaseDate: String? =
    null,
13    @SerialName("vote_average") val voteAverage: Double? =
    null,
14        @SerialName("original_title") val originalTitle:
    String? = null
15 )
```

*Tabel 4. Source Code Resource.kt Modul 5*

```
1 package com.mobil.modul5compose.data
2
3 sealed interface Resource<out T> {
4     data class Success<T>(val data: T) : Resource<T>
5     data class Error<T>(val exception: Throwable, val data:
    T? = null) : Resource<T>
6     data class Loading<T>(val data: T? = null) : Resource<T>
7 }
```

*Tabel 5. Source Code SettingsRepository.kt Modul 5*

```
1 package com.mobil.modul5compose.data
2
3 import android.content.Context
4 import android.content.SharedPreferences
5 import
```

```
    com.mobil.modul5compose.ui.screens.language.LanguageModel
6   import timber.log.Timber
7   import kotlinx.coroutines.flow.MutableStateFlow
8   import kotlinx.coroutines.flow.StateFlow
9   import kotlinx.coroutines.flow.asStateFlow
10
11  class SettingsRepository(context: Context) {
12              private val prefs: SharedPreferences =
    context.getSharedPreferences(PREFS_NAME,
    Context.MODE_PRIVATE)
13
14                      private val _languageTag =
    MutableStateFlow(getLanguageTag())
15                  val languageTag: StateFlow<String> =
    _languageTag.asStateFlow()
16
17      fun getLanguageTag(): String {
18              return prefs.getString(KEY_LANGUAGE, "en-US") ?:
    "en-US"
19      }
20
21      fun saveLanguageTag(tag: String) {
22          Timber.d("Saving language tag to SharedPreferences:
    %s", tag)
23          prefs.edit().putString(KEY_LANGUAGE, tag).apply()
24          _languageTag.value = tag
25      }
26
27      fun getAvailableLanguages(): List<LanguageModel> {
```

```
28          return defaultLanguages()
29      }
30
31      private fun defaultLanguages() = listOf(
32          LanguageModel("English", "en-US"),
33          LanguageModel("Bahasa Indonesia", "id-ID"),
34          LanguageModel("Español", "es-ES"),
35          LanguageModel("Français", "fr-FR"),
36          LanguageModel("Deutsch", "de-DE")
37      )
38
39      companion object {
40          private const val PREFS_NAME = "app_settings"
41          private const val KEY_LANGUAGE = "language_tag"
42      }
43  }
```

*Tabel 6. Source Code TmdbListResponse.kt Modul 5*

```
1  package com.mobil.modul5compose.data
2
3  import kotlinx.serialization.Serializable
4
5  @Serializable
6  data class TmdbListResponse(
7      val results: List<MovieResponse>
8  )
```

### 3. Data.Local

*Tabel 7. Source Code MovieEntity.kt Modul 5*

```
1  package com.mobil.modul5compose.data.local
2
3  import androidx.room.Entity
4  import androidx.room.PrimaryKey
5
6  @Entity(tableName = "movies")
7  data class MovieEntity(
8      @PrimaryKey
9      val id: Int,
10     val title: String,
11     val overview: String,
12     val posterPath: String,
13     val releaseDate: String,
14     val voteAverage: Double,
15     val languageTag: String,
16     val cachedAt: Long = System.currentTimeMillis()
17 )
```

*Tabel 8. Source Code MovieDao.kt Modul 5*

```
1  package com.mobil.modul5compose.data.local
2
3  import androidx.room.Dao
4  import androidx.room.Insert
5  import androidx.room.OnConflictStrategy
6  import androidx.room.Query
```

```
7  import kotlinx.coroutines.flow.Flow
8
9  @Dao
10 interface MovieDao {
11         @Query("SELECT * FROM movies WHERE languageTag
   = :languageTag ORDER BY cachedAt DESC")
12                 fun getAllMovies(languageTag: String):
   Flow<List<MovieEntity>>
13
14      @Query("SELECT * FROM movies WHERE id = :movieId AND
   languageTag = :languageTag")
15      fun getMovieById(movieId: Int, languageTag: String):
   Flow<MovieEntity>
16
17     @Insert(onConflict = OnConflictStrategy.REPLACE)
18     suspend fun insertMovie(movie: MovieEntity)
19
20     @Insert(onConflict = OnConflictStrategy.REPLACE)
21     suspend fun insertMovie(movie: List<MovieEntity>)
22
23     @Query("DELETE FROM movies")
24     suspend fun clearAllMovies()
25 }
```

**4. Navigation**

*Tabel 9. Source Code AppDatabase.kt Modul 5*

```
1  package com.mobil.modul5compose.data.local
2
```

```
3   import androidx.room.Database
4   import androidx.room.RoomDatabase
5
6   @Database(
7       entities = [MovieEntity::class],
8       version = 1,
9       exportSchema = false
10  )
11  abstract class AppDatabase: RoomDatabase() {
12      abstract fun movieDao(): MovieDao
13  }
```

*Tabel 10. Source Code RoutingNames.kt Modul 5*

```
1   package com.mobil.modul5compose.navigation
2
3   import kotlinx.serialization.Serializable
4
5   @Serializable
6   sealed class RoutingNames {
7       @Serializable
8       object HomeScreen : RoutingNames()
9
10      @Serializable
11      data class DetailScreen(
12          val movieId: Int
13      ) : RoutingNames()
14
```

```
15      @Serializable
16      object LanguageScreen: RoutingNames()
17  }
```

*Tabel 11. Source Code AppNavigation.kt Modul 5*

```
1   package com.mobil.modul5compose.navigation
2
3   import androidx.compose.runtime.Composable
4   import androidx.compose.runtime.remember
5   import androidx.lifecycle.viewmodel.compose.viewModel
6   import androidx.navigation.NavHostController
7   import androidx.navigation.compose.NavHost
8   import androidx.navigation.compose.composable
9   import androidx.navigation.toRoute
10  import com.mobil.modul5compose.data.SettingsRepository
11  import com.mobil.modul5compose.data.local.MovieDao
12  import com.mobil.modul5compose.network.MovieRepository
13  import
    com.mobil.modul5compose.ui.screens.detail.DetailScreen
14  import
    com.mobil.modul5compose.ui.screens.detail.DetailViewModel
15  import
    com.mobil.modul5compose.ui.screens.detail.DetailViewModelFa
    ctory
16  import com.mobil.modul5compose.ui.screens.home.HomeScreen
17  import
    com.mobil.modul5compose.ui.screens.home.HomeViewModel
18  import
    com.mobil.modul5compose.ui.screens.home.HomeViewModelFactor
```

```
y
import com.mobil.modul5compose.ui.screens.language.LanguageScreen
import com.mobil.modul5compose.ui.screens.language.LanguageViewModel
import com.mobil.modul5compose.ui.screens.language.LanguageViewModelFactory

@Composable
fun AppNavigation(
    navController: NavHostController,
    settingsRepository: SettingsRepository,
    movieRepository: MovieRepository
) {
    NavHost(navController = navController, startDestination = RoutingNames.HomeScreen) {
        composable<RoutingNames.HomeScreen> {
                                val factory = remember { HomeViewModelFactory(movieRepository) }
                            val viewModel: HomeViewModel = viewModel(factory = factory)
            HomeScreen(navController, viewModel)
        }
                        composable<RoutingNames.DetailScreen> { navBackStack ->
                                        val detailArgs = navBackStack.toRoute<RoutingNames.DetailScreen>()
                                val factory = remember
```

```
    {                    DetailViewModelFactory(movieRepository,
    detailArgs.movieId) }
38                          val  viewModel:  DetailViewModel  =
    viewModel(factory = factory)
39                                      DetailScreen(viewModel)
    { navController.popBackStack() }
40          }
41          composable<RoutingNames.LanguageScreen> {
42                                    val   factory   =   remember
    { LanguageViewModelFactory(settingsRepository) }
43                        val  viewModel:  LanguageViewModel  =
    viewModel(factory = factory)
44              LanguageScreen(navController, viewModel)
45          }
46      }
47  }
```

**5. Network**

*Tabel 12. Source Code HttpClientFactory.kt Modul 5*

```
1   package com.mobil.modul5compose.network
2
3   import io.ktor.client.HttpClient
4
5   interface HttpClientFactory {
6       fun create(): HttpClient
7   }
```

*Tabel 13. Source Code TmdbHttpClientFactory.kt Modul 5*

```
1  package com.mobil.modul5compose.network
2
3  import io.ktor.client.HttpClient
4  import io.ktor.client.engine.cio.CIO
5  import
   io.ktor.client.plugins.contentnegotiation.ContentNegotiatio
   n
6  import io.ktor.client.plugins.defaultRequest
7  import io.ktor.client.request.header
8  import io.ktor.http.ContentType
9  import io.ktor.http.HttpHeaders
10 import io.ktor.serialization.kotlinx.json.json
11 import kotlinx.serialization.json.Json
12
13 class TmdbHttpClientFactory(
14     private val apiKey: String
15 ) : HttpClientFactory {
16     override fun create(): HttpClient {
17         return HttpClient(CIO) {
18             install(ContentNegotiation){
19                 json(Json{
20                     ignoreUnknownKeys = true
21                     coerceInputValues = true
22                 })
23             }
24
25             defaultRequest {
```

```
                url("https://api.themoviedb.org/3/")
                  header(HttpHeaders.Authorization, "Bearer $apiKey")
                              header(HttpHeaders.ContentType, ContentType.Application.Json)
            }
        }
    }
}
```

*Tabel 14. Source Code BaseRepository.kt Modul 5*

```
package com.mobil.modul5compose.network

import io.ktor.client.call.body
import io.ktor.client.statement.HttpResponse
import io.ktor.client.statement.bodyAsText
import io.ktor.http.isSuccess
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.flow
import kotlinx.coroutines.flow.flowOn
import timber.log.Timber

abstract class BaseRepository {
    protected suspend inline fun <reified T> safeApiFlow(
        crossinline call: suspend () -> HttpResponse
    ): Flow<Result<T>> = flow<Result<T>> {
        val result = runCatching {
```

```
18              val response = call()
19              if (response.status.isSuccess()) {
20                  response.body<T>()
21              } else {
22                  val errorBody = response.bodyAsText()
23                              val errorMessage = "Status: ${response.status.value} | Body: $errorBody"
24
25                          Timber.tag("Network").e("API Failure: $errorMessage")
26                              throw Exception("API Error: ${response.status.value}")
27              }
28          }.onFailure { exception ->
29                  Timber.tag("Network").e(exception, "Network Failure")
30          }
31      }.flowOn(Dispatchers.IO)
32  }
```

*Tabel 15. Source Code MovieRepository.kt Modul 5*

```
1  package com.mobil.modul5compose.network
2
3  import com.mobil.modul5compose.data.MovieResponse
4  import com.mobil.modul5compose.data.Resource
5  import com.mobil.modul5compose.data.SettingsRepository
6  import com.mobil.modul5compose.data.TmdbListResponse
```

```
7  import com.mobil.modul5compose.data.local.MovieDao
8  import com.mobil.modul5compose.data.local.MovieEntity
9  import com.mobil.modul5compose.util.toEntity
10 import io.ktor.client.HttpClient
11 import io.ktor.client.call.body
12 import io.ktor.client.request.get
13 import io.ktor.http.isSuccess
14 import kotlinx.coroutines.Dispatchers
15 import kotlinx.coroutines.flow.Flow
16 import kotlinx.coroutines.flow.filterNotNull
17 import kotlinx.coroutines.flow.flow
18 import kotlinx.coroutines.flow.flowOn
19
20 class MovieRepository(
21     private val client: HttpClient,
22     private val movieDao: MovieDao,
23     private val settingsRepository: SettingsRepository
24 ) : BaseRepository() {
25
26     val languageTag = settingsRepository.languageTag
27
28
29                                 fun        getPopularMovies():
    Flow<Resource<List<MovieEntity>>> {
30                                   val    currentLanguage    =
    settingsRepository.getLanguageTag()
31         return networkBoundResource(
32             query = {
```

```
33                movieDao.getAllMovies(currentLanguage)
34            },
35            fetch = {
36                  val response = client.get("movie/popular?
language=$currentLanguage")
37                if(!response.status.isSuccess()){
38                                throw Exception("API Error: $
{response.status.value}")
39                }
40                response.body<TmdbListResponse>()
41            },
42            saveFetchResult = { response ->
43                movieDao.insertMovie(response.results.map {
it.toEntity(currentLanguage) })
44            },
45            shouldFetch = { cachedList ->
46                cachedList.isNullOrEmpty()
47            }
48        ).flowOn(Dispatchers.IO)
49    }
50
51    fun getMovie(movieId: Int): Flow<Resource<MovieEntity>>
{
52                                val    currentLanguage    =
settingsRepository.getLanguageTag()
53        return networkBoundResource(
54            query = {
55                                movieDao.getMovieById(movieId,
currentLanguage).filterNotNull()
```

```
56              },
57              fetch = {
58                   val response = client.get("movie/$movieId?
language=$currentLanguage")
59                  if(!response.status.isSuccess()){
60                                  throw Exception("API Error: $
{response.status.value}")
61                  }
62                  response.body<MovieResponse>()
63              },
64              saveFetchResult = { response ->
65                  movieDao.insertMovie(response.toEntity(curr
entLanguage))
66              },
67              shouldFetch = { cachedEntity ->
68                  cachedEntity == null
69              }
70          ).flowOn(Dispatchers.IO)
71      }
72
73
74  }
```

*Tabel 16. Source Code NetworkBoundResource.kt Modul 5*

```
1  package com.mobil.modul5compose.network
2
3  import com.mobil.modul5compose.data.Resource
4  import kotlinx.coroutines.flow.Flow
```

```
5  import kotlinx.coroutines.flow.emitAll
6  import kotlinx.coroutines.flow.firstOrNull
7  import kotlinx.coroutines.flow.flow
8  import kotlinx.coroutines.flow.map
9
10 inline fun <ResultType, RequestType> networkBoundResource(
11     crossinline query: () -> Flow<ResultType>,
12     crossinline fetch: suspend () -> RequestType,
13      crossinline saveFetchResult: suspend (RequestType) ->
   Unit,
14       crossinline shouldFetch: (ResultType?) -> Boolean =
   { true }
15 ): Flow<Resource<ResultType>> = flow {
16
17     val data = query().firstOrNull()
18
19     if (shouldFetch(data)) {
20         emit(Resource.Loading(data))
21
22         try {
23             val fetchResult = fetch()
24             saveFetchResult(fetchResult)
25             emitAll(query().map { Resource.Success(it) })
26         } catch (throwable: Throwable) {
27             emitAll(query().map { Resource.Error(throwable,
   it) })
28         }
29     } else {
```

```
30          emitAll(query().map { Resource.Success(it!!) })
31      }
32  }
```

**6. UI.Screens.Detail**

*Tabel 17. Source Code DetailScreen.kt Modul 5*

```
1   package com.mobil.modul5compose.ui.screens.detail
2
3   import androidx.compose.foundation.layout.Arrangement
4   import androidx.compose.foundation.layout.Box
5   import androidx.compose.foundation.layout.Column
6   import androidx.compose.foundation.layout.Row
7   import androidx.compose.foundation.layout.Spacer
8   import androidx.compose.foundation.layout.fillMaxSize
9   import androidx.compose.foundation.layout.fillMaxWidth
10  import androidx.compose.foundation.layout.height
11  import androidx.compose.foundation.layout.padding
12  import androidx.compose.foundation.rememberScrollState
13  import
    androidx.compose.foundation.shape.RoundedCornerShape
14  import androidx.compose.foundation.verticalScroll
15  import androidx.compose.material.icons.Icons
16  import
    androidx.compose.material.icons.automirrored.filled.ArrowB
    ack
17  import
    androidx.compose.material3.CircularProgressIndicator
18  import androidx.compose.material3.ExperimentalMaterial3Api
```

```
19 import androidx.compose.material3.Icon
20 import androidx.compose.material3.IconButton
21 import androidx.compose.material3.MaterialTheme
22 import androidx.compose.material3.Scaffold
23 import androidx.compose.material3.Text
24 import androidx.compose.material3.TopAppBar
25 import androidx.compose.runtime.Composable
26 import androidx.compose.runtime.collectAsState
27 import androidx.compose.runtime.getValue
28 import androidx.compose.ui.Alignment
29 import androidx.compose.ui.Modifier
30 import androidx.compose.ui.draw.clip
31 import androidx.compose.ui.layout.ContentScale
32 import androidx.compose.ui.text.font.FontWeight
33 import androidx.compose.ui.unit.dp
34 import coil.compose.AsyncImage
35
36 @OptIn(ExperimentalMaterial3Api::class)
37 @Composable
38 fun DetailScreen(
39     viewModel: DetailViewModel,
40     onBack: () -> Unit
41 ) {
42     val state by viewModel.state.collectAsState()
43
44     Scaffold(
45         topBar = {
46             TopAppBar(
```

```
                    title = { Text(state.movie?.title ?: "Loading...") },
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Back")
                    }
                }
            )
        }
    ) { paddingValues ->
        Box(
            modifier = Modifier
                .padding(paddingValues)
                .fillMaxSize()
        ) {

            if (state.isLoading && state.movie == null) {
                    CircularProgressIndicator(modifier = Modifier.align(Alignment.Center))
            }

            state.movie?.let { movie ->
                Column(
                    modifier = Modifier
                        .fillMaxSize()
                        .verticalScroll(rememberScrollState())
                        .padding(16.dp)
```

```
72                ) {
73                    AsyncImage(
74                                                    model =
    "https://image.tmdb.org/t/p/w500${movie.posterPath}",
75                                    contentDescription = "$
    {movie.title} Poster",
76                        modifier = Modifier
77                            .fillMaxWidth()
78                            .height(450.dp)
79                            .clip(RoundedCornerShape(12.dp
    )),
80                        contentScale = ContentScale.Crop
81                    )
82
83                                        Spacer(modifier =
    Modifier.height(16.dp))
84
85                    Text(
86                        text = movie.title,
87                                                    style =
    MaterialTheme.typography.headlineMedium,
88                        fontWeight = FontWeight.Bold
89                    )
90
91                                        Spacer(modifier =
    Modifier.height(8.dp))
92
93                    Row(
```

```
94                                                  modifier =
    Modifier.fillMaxWidth(),
95                                          horizontalArrangement =
    Arrangement.SpaceBetween
96                      ) {
97                          Text(
98                                                text = "Release: $
    {movie.releaseDate}",
99                                                    style =
    MaterialTheme.typography.bodyMedium,
100                                                   color =
    MaterialTheme.colorScheme.secondary
101                         )
102                         Text(
103                                                text = "★ $
    {movie.voteAverage}/10",
104                                                   style =
    MaterialTheme.typography.bodyMedium,
105                                                   color =
    MaterialTheme.colorScheme.secondary,
106                             fontWeight = FontWeight.Bold
107                         )
108                     }
109
110                                         Spacer(modifier =
    Modifier.height(24.dp))
111
112                     Text(
113                         text = "Overview",
114                                                   style =
```

```
    MaterialTheme.typography.titleMedium,
115                         fontWeight = FontWeight.SemiBold
116                     )
117
118                                         Spacer(modifier =
    Modifier.height(8.dp))
119
120                     Text(
121                         text = movie.overview,
122                                                 style =
    MaterialTheme.typography.bodyLarge
123                     )
124                 }
125             }
126
127             state.errorMessage?.let { error ->
128                 Text(
129                     text = error,
130                                                 color =
    MaterialTheme.colorScheme.error,
131                     modifier = Modifier
132                         .align(Alignment.Center)
133                         .padding(16.dp)
134                 )
135             }
136         }
137     }
138 }
```

*Tabel 18. Source Code DetailState.kt Modul 5*

```
1  package com.mobil.modul5compose.ui.screens.detail
2  import com.mobil.modul5compose.data.local.MovieEntity
3
4  data class DetailState(
5      val movie: MovieEntity? = null,
6      val isLoading: Boolean = false,
7      val errorMessage: String? = null
8  )
```

*Tabel 19. Source Code DetailViewModel.kt Modul 5*

```
1  package com.mobil.modul5compose.ui.screens.detail
2
3  import androidx.lifecycle.ViewModel
4  import androidx.lifecycle.viewModelScope
5  import com.mobil.modul5compose.data.Resource
6  import com.mobil.modul5compose.network.MovieRepository
7  import kotlinx.coroutines.flow.MutableStateFlow
8  import kotlinx.coroutines.flow.StateFlow
9  import kotlinx.coroutines.flow.asStateFlow
10 import kotlinx.coroutines.flow.update
11 import kotlinx.coroutines.launch
12
13 class DetailViewModel(
14     private val repository: MovieRepository,
15     private val movieId: Int
```

```
16 ) : ViewModel() {
17     private val _state = MutableStateFlow(DetailState())
18                 val    state:    StateFlow<DetailState>    =
   _state.asStateFlow()
19
20     init { loadMovie() }
21
22     private fun loadMovie() {
23         viewModelScope.launch {
24                repository.getMovie(movieId).collect { result
   ->
25                  when (result) {
26                          is Resource.Loading -> _state.update
   {  it.copy(isLoading  =  true,  movie  =  result.data  ?:
   it.movie) }
27                          is Resource.Success -> _state.update
   {  it.copy(isLoading  =  false,  movie  =  result.data,
   errorMessage = null) }
28                            is Resource.Error -> _state.update
   {    it.copy(isLoading    =    false,    errorMessage    =
   result.exception.message,    movie    =    result.data    ?:
   it.movie) }
29                  }
30              }
31         }
32     }
33 }
```

*Tabel 20. Source Code DetailViewModelFactory.kt Modul 5*

```
1  package com.mobil.modul5compose.ui.screens.detail
2
3  import androidx.lifecycle.ViewModel
4  import androidx.lifecycle.ViewModelProvider
5  import com.mobil.modul5compose.network.MovieRepository
6
7  class DetailViewModelFactory(
8      private val repository: MovieRepository,
9      private val movieId: Int
10 ) : ViewModelProvider.Factory {
11         override fun <T : ViewModel> create(modelClass:
   Class<T>): T {
12                                                      if
   (modelClass.isAssignableFrom(DetailViewModel::class.java))
   {
13             return DetailViewModel(repository, movieId) as
   T
14         }
15           throw IllegalArgumentException("Unknown ViewModel
   class")
16     }
17 }
```

**7. UI.Screens.Home**

*Tabel 21. Source Code HomeState.kt Modul 5*

```
1  package com.mobil.modul5compose.ui.screens.home
2
3  import com.mobil.modul5compose.data.local.MovieEntity
```

```
4
5  data class HomeState(
6      val movies: List<MovieEntity> = emptyList(),
7      val isLoading: Boolean = false,
8      val errorMessage: String? = null,
9      val navigateToDetailEvent: Int? = null,
10     val openExternalUrlEvent: String? = null
11 )
```

*Tabel 22. Source Code HomeViewModel.kt Modul 5*

```
1  package com.mobil.modul5compose.ui.screens.home
2
3  import androidx.lifecycle.ViewModel
4  import androidx.lifecycle.viewModelScope
5  import com.mobil.modul5compose.data.Resource
6  import com.mobil.modul5compose.network.MovieRepository
7  import kotlinx.coroutines.flow.MutableStateFlow
8  import kotlinx.coroutines.flow.StateFlow
9  import kotlinx.coroutines.flow.asStateFlow
10 import kotlinx.coroutines.flow.update
11 import kotlinx.coroutines.launch
12 import kotlinx.coroutines.ExperimentalCoroutinesApi
13
14 @OptIn(ExperimentalCoroutinesApi::class)
15 class HomeViewModel(private val repository:
   MovieRepository) : ViewModel() {
16     private val _uiState = MutableStateFlow(HomeState())
```

```
17                val uiState: StateFlow<HomeState> =
_uiState.asStateFlow()
18
19    init { loadMovies() }
20
21    private fun loadMovies() {
22        viewModelScope.launch {
23            repository.languageTag.collect {
24                loadMoviesForCurrentLanguage()
25            }
26        }
27    }
28
29    private var moviesJob: kotlinx.coroutines.Job? = null
30
31    private fun loadMoviesForCurrentLanguage() {
32        moviesJob?.cancel()
33        moviesJob = viewModelScope.launch {
34             repository.getPopularMovies().collect { result
->
35                when (result) {
36                    is Resource.Loading -> _uiState.update
{ it.copy(isLoading = true, movies = result.data ?:
it.movies) }
37                    is Resource.Success -> _uiState.update
{ it.copy(isLoading = false, movies = result.data,
errorMessage = null) }
38                     is Resource.Error -> _uiState.update
{   it.copy(isLoading   =   false,   errorMessage   =
result.exception.message,   movies   =   result.data   ?:
```

```
    it.movies) }
39                  }
40              }
41          }
42      }
43
44      fun onMovieClicked(movieId: Int) {
45              _uiState.update { it.copy(navigateToDetailEvent =
    movieId) }
46      }
47
48      fun onDetailNavigationHandled() {
49              _uiState.update { it.copy(navigateToDetailEvent =
    null) }
50      }
51
52      fun onExternalUrlClicked(url: String) {
53              _uiState.update { it.copy(openExternalUrlEvent =
    url) }
54      }
55
56      fun onExternalUrlHandled() {
57              _uiState.update { it.copy(openExternalUrlEvent =
    null) }
58      }
59  }
```

*Tabel 23. Source Code HomeScreen.kt Modul 5*

```
1   package com.mobil.modul5compose.ui.screens.home
2
3   import android.content.Intent
4   import androidx.compose.foundation.Image
5   import androidx.compose.foundation.clickable
6   import androidx.compose.foundation.layout.Arrangement
7   import androidx.compose.foundation.layout.Box
8   import androidx.compose.foundation.layout.Column
9   import androidx.compose.foundation.layout.PaddingValues
10  import androidx.compose.foundation.layout.Row
11  import androidx.compose.foundation.layout.Spacer
12  import androidx.compose.foundation.layout.fillMaxSize
13  import androidx.compose.foundation.layout.fillMaxWidth
14  import androidx.compose.foundation.layout.height
15  import androidx.compose.foundation.layout.padding
16  import androidx.compose.foundation.layout.width
17  import
    androidx.compose.foundation.layout.wrapContentHeight
18  import androidx.compose.foundation.lazy.LazyColumn
19  import androidx.compose.foundation.lazy.items
20  import
    androidx.compose.foundation.shape.RoundedCornerShape
21  import androidx.compose.material.icons.Icons
22  import androidx.compose.material.icons.filled.Language
23  import
    androidx.compose.material.icons.filled.OpenInBrowser
24  import androidx.compose.material3.Button
```

```
25 import androidx.compose.material3.Card
26 import androidx.compose.material3.CardDefaults
27 import
   androidx.compose.material3.CircularProgressIndicator
28 import androidx.compose.material3.ExperimentalMaterial3Api
29 import androidx.compose.material3.Icon
30 import androidx.compose.material3.IconButton
31 import androidx.compose.material3.MaterialTheme
32 import androidx.compose.material3.Scaffold
33 import androidx.compose.material3.Text
34 import androidx.compose.material3.TopAppBar
35 import
   androidx.compose.material3.carousel.HorizontalMultiBrowseC
   arousel
36 import
   androidx.compose.material3.carousel.rememberCarouselState
37 import androidx.compose.runtime.Composable
38 import androidx.compose.runtime.LaunchedEffect
39 import androidx.compose.runtime.collectAsState
40 import androidx.compose.runtime.getValue
41 import androidx.compose.ui.Alignment
42 import androidx.compose.ui.Modifier
43 import androidx.compose.ui.draw.clip
44 import androidx.compose.ui.layout.ContentScale
45 import androidx.compose.ui.platform.LocalContext
46 import androidx.compose.ui.res.painterResource
47 import androidx.compose.ui.res.stringResource
48 import androidx.compose.ui.text.font.FontWeight
49 import androidx.compose.ui.text.style.TextAlign
```

```
50 import androidx.compose.ui.text.style.TextOverflow
51 import androidx.compose.ui.unit.dp
52 import androidx.compose.ui.unit.sp
53 import androidx.navigation.NavController
54 import androidx.core.net.toUri
55 import coil.compose.AsyncImage
56 import com.mobil.modul5compose.R
57 import com.mobil.modul5compose.data.local.MovieEntity
58 import com.mobil.modul5compose.navigation.RoutingNames
59 import timber.log.Timber
60
61 @OptIn(ExperimentalMaterial3Api::class)
62 @Composable
63 fun HomeScreen(
64     navController: NavController,
65     viewModel: HomeViewModel
66 ) {
67     val state by viewModel.uiState.collectAsState()
68     val context = LocalContext.current
69
70     LaunchedEffect(state.navigateToDetailEvent) {
71         state.navigateToDetailEvent?.let { movieId ->
72                 Timber.d("Navigating to detail page for
    movieId: $movieId")
73             navController.navigate(RoutingNames.DetailScre
    en(movieId))
74             viewModel.onDetailNavigationHandled()
```

```
75          }
76      }
77
78      LaunchedEffect(state.openExternalUrlEvent) {
79          state.openExternalUrlEvent?.let { url ->
80              Timber.d("Opening external URL: $url")
81              context.startActivity(Intent(Intent.ACTION_VIE
    W, url.toUri()))
82              viewModel.onExternalUrlHandled()
83          }
84      }
85
86      Scaffold(
87          topBar = {
88              TopAppBar(
89                          title = { Text(stringResource(id =
    R.string.app_name)) },
90                  actions = {
91                                          IconButton(onClick =
    { navController.navigate(RoutingNames.LanguageScreen) }) {
92                          Icon(
93                                                  imageVector =
    Icons.Default.Language,
94                                  contentDescription = "Change
    Language"
95                          )
96                      }
97                  }
98              )
```

```
99          }
100     ) { paddingValues ->
101         Box(
102             modifier = Modifier
103                 .padding(paddingValues)
104                 .fillMaxSize()
105         ) {
106             if (state.isLoading && state.movies.isEmpty())
    {
107                     CircularProgressIndicator(modifier =
    Modifier.align(Alignment.Center))
108             } else {
109                 HomeContent(
110                     movies = state.movies,
111                                             onExternalClick  =
    viewModel::onExternalUrlClicked,
112                                               onDetailClick  =
    viewModel::onMovieClicked
113                 )
114             }
115
116             state.errorMessage?.let { error ->
117                 Text(
118                     text = error,
119                                                       color  =
    MaterialTheme.colorScheme.error,
120                     modifier = Modifier
121                         .align(Alignment.BottomCenter)
122                         .padding(16.dp)
```

```
                )
            }
        }
    }
}

@Composable
private fun HomeContent(
    movies: List<MovieEntity>,
    onExternalClick: (String) -> Unit,
    onDetailClick: (Int) -> Unit,
    modifier: Modifier = Modifier
) {
    LazyColumn(
        modifier = modifier.fillMaxSize(),
        contentPadding = PaddingValues(bottom = 16.dp)
    ) {
        if (movies.isNotEmpty()) {
            item {
                MovieCarousel(
                    movies = movies.take(5),
                    onItemClick = onDetailClick
                )
            }

            items(
                items = movies,
                key = { it.id }
```

```
151             ) { movie ->
152                 MovieCard(
153                     movie = movie,
154                                             onExternalClick  =
    {       onExternalClick("https://www.themoviedb.org/movie/$
    {movie.id}") },
155                                               onDetailClick  =
    { onDetailClick(movie.id) }
156                 )
157             }
158         }
159     }
160 }
161
162 @Composable
163 private fun MovieCard(
164     movie: MovieEntity,
165     onExternalClick: () -> Unit,
166     onDetailClick: () -> Unit
167 ) {
168     Card(
169          colors = CardDefaults.cardColors(containerColor =
    MaterialTheme.colorScheme.surfaceVariant),
170         modifier = Modifier
171             .padding(horizontal = 16.dp, vertical = 8.dp)
172             .fillMaxWidth()
173             .clickable { onDetailClick() }
174     ) {
175         Row(
```

```
176             verticalAlignment = Alignment.Top,
177             modifier = Modifier.padding(8.dp)
178         ) {
179             AsyncImage(
180                  model = "https://image.tmdb.org/t/p/w500${movie.posterPath}",
181                      contentDescription = "${movie.title} Poster",
182                 contentScale = ContentScale.Crop,
183                 modifier = Modifier
184                     .width(100.dp)
185                     .height(150.dp)
186                     .clip(RoundedCornerShape(8.dp))
187             )
188 
189             Spacer(modifier = Modifier.width(16.dp))
190 
191             Column(
192                 modifier = Modifier.weight(1f)
193             ) {
194                 Row(
195                     modifier = Modifier.fillMaxWidth(),
196                                     horizontalArrangement = Arrangement.SpaceBetween,
197                     verticalAlignment = Alignment.Top
198                 ) {
199                     Text(
```

```
200                         text = movie.title,
201                         fontWeight = FontWeight.Bold,
202                                                 style  =
    MaterialTheme.typography.titleMedium,
203                         maxLines = 2,
204                         overflow = TextOverflow.Ellipsis,
205                         modifier = Modifier.weight(1f)
206                     )
207                     IconButton(
208                         onClick = onExternalClick,
209                          modifier = Modifier.padding(start
    = 8.dp)
210                     ) {
211                         Icon(
212                                                 imageVector  =
    Icons.Default.OpenInBrowser,
213                                 contentDescription = "Open in
    TMDB",
214                                                  tint  =
    MaterialTheme.colorScheme.primary
215                         )
216                     }
217                 }
218
219                 Text(
220                     text = "★ ${movie.voteAverage}",
221                                                 style  =
    MaterialTheme.typography.bodySmall,
222                                                 color  =
```

```
    MaterialTheme.colorScheme.secondary,
223                     modifier = Modifier.padding(vertical =
    4.dp)
224                 )
225
226                 Text(
227                     text = movie.overview,
228                     maxLines = 3,
229                     overflow = TextOverflow.Ellipsis,
230                                                 style   =
    MaterialTheme.typography.bodyMedium
231                 )
232             }
233         }
234     }
235 }
236
237 @OptIn(ExperimentalMaterial3Api::class)
238 @Composable
239 private fun MovieCarousel(
240     movies: List<MovieEntity>,
241     onItemClick: (Int) -> Unit
242 ) {
243     HorizontalMultiBrowseCarousel(
244         state = rememberCarouselState { movies.count() },
245         modifier = Modifier
246             .fillMaxWidth()
247             .wrapContentHeight()
```

```
            .padding(top = 16.dp, bottom = 16.dp),
        preferredItemWidth = 372.dp,
        itemSpacing = 8.dp,
        contentPadding = PaddingValues(horizontal = 16.dp)
    ) { i ->
        val movie = movies[i]

        AsyncImage(
                model = "https://image.tmdb.org/t/p/w500${movie.posterPath}",
            contentDescription = movie.title,
            contentScale = ContentScale.Crop,
            modifier = Modifier
                .height(300.dp)
                .maskClip(MaterialTheme.shapes.extraLarge)
                .clickable { onItemClick(movie.id) }
                .fillMaxSize()
        )
    }
}
```

*Tabel 24. Source Code HomeViewModelFactory.kt Modul 5*

```
package com.mobil.modul5compose.ui.screens.home

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import com.mobil.modul5compose.network.MovieRepository
```

```
6
7  class HomeViewModelFactory(
8      private val repository: MovieRepository
9  ) : ViewModelProvider.Factory {
10     @Suppress("UNCHECKED_CAST")
11         override fun <T : ViewModel> create(modelClass:
   Class<T>): T {
12                                                     if
   (modelClass.isAssignableFrom(HomeViewModel::class.java)) {
13             return HomeViewModel(repository) as T
14         }
15           throw IllegalArgumentException("Unknown ViewModel
   class")
16     }
17 }
```

**8. UI.Screens.Language**

*Tabel 25. Source Code LanguageScreen.kt Modul 5*

```
1  package com.mobil.modul5compose.ui.screens.language
2
3  import androidx.compose.foundation.clickable
4  import androidx.compose.foundation.layout.Arrangement
5  import androidx.compose.foundation.layout.Row
6  import androidx.compose.foundation.layout.fillMaxSize
7  import androidx.compose.foundation.layout.fillMaxWidth
8  import androidx.compose.foundation.layout.padding
9  import androidx.compose.foundation.lazy.LazyColumn
10 import androidx.compose.foundation.lazy.items
```

```
11 import androidx.compose.material.icons.Icons
12 import
   androidx.compose.material.icons.automirrored.filled.ArrowB
   ack
13 import androidx.compose.material.icons.filled.Check
14 import androidx.compose.material3.ExperimentalMaterial3Api
15 import androidx.compose.material3.Icon
16 import androidx.compose.material3.IconButton
17 import androidx.compose.material3.Scaffold
18 import androidx.compose.material3.Text
19 import androidx.compose.material3.TopAppBar
20 import androidx.compose.runtime.Composable
21 import androidx.compose.runtime.LaunchedEffect
22 import androidx.compose.runtime.collectAsState
23 import androidx.compose.runtime.getValue
24 import androidx.compose.ui.Alignment
25 import androidx.compose.ui.Modifier
26 import androidx.compose.ui.res.stringResource
27 import androidx.compose.ui.unit.dp
28 import androidx.navigation.NavController
29 import com.mobil.modul5compose.R
30
31 @Composable
32 fun LanguageScreen(
33     navController: NavController,
34     viewModel: LanguageViewModel
35 ) {
36     val state by viewModel.uiState.collectAsState()
```

```
37
38      LaunchedEffect(state.navigateBackEvent) {
39          if (state.navigateBackEvent) {
40              navController.popBackStack()
41              viewModel.onNavigateBackHandled()
42          }
43      }
44
45      LanguageContent(
46          state = state,
47          onBackClick = { navController.popBackStack() },
48                                          onLanguageClick     =
    { viewModel.onLanguageSelected(it) }
49      )
50  }
51
52  @OptIn(ExperimentalMaterial3Api::class)
53  @Composable
54  private fun LanguageContent(
55      state: LanguageState,
56      onBackClick: () -> Unit,
57      onLanguageClick: (String) -> Unit
58  ) {
59      Scaffold(
60          topBar = {
61              TopAppBar(
62                          title = { Text(stringResource(id =
    R.string.language_title)) },
```

```
63                 navigationIcon = {
64                     IconButton(onClick = onBackClick) {
65                         Icon(Icons.AutoMirrored.Filled.Arr
owBack, contentDescription = "Back")
66                     }
67                 }
68             )
69         }
70     ) { paddingValues ->
71         LazyColumn(
72             modifier = Modifier
73                 .fillMaxSize()
74                 .padding(paddingValues)
75         ) {
76             items(state.languages) { language ->
77                 LanguageItem(
78                     title = language.name,
79                     tag = language.tag,
80                     currentTag = state.selectedTag,
81                                                 onClick  =
{ onLanguageClick(language.tag) }
82                 )
83             }
84         }
85     }
86 }
87
88 @Composable
```

```
89  private fun LanguageItem(
90      title: String,
91      tag: String,
92      currentTag: String,
93      onClick: () -> Unit
94  ) {
95      Row(
96          modifier = Modifier
97              .fillMaxWidth()
98              .clickable(onClick = onClick)
99              .padding(16.dp),
100         horizontalArrangement = Arrangement.SpaceBetween,
101         verticalAlignment = Alignment.CenterVertically
102     ) {
103         Text(text = title)
104         if (currentTag == tag) {
105             Icon(Icons.Default.Check, contentDescription =
    "Selected")
106         }
107     }
108 }
```

*Tabel 26. Source Code LanguageState.kt Modul 5*

```
1  package com.mobil.modul5compose.ui.screens.language
2
3  data class LanguageState(
4      val selectedTag: String = "en-US",
```

```
5       val languages: List<LanguageModel> = emptyList(),
6       val navigateBackEvent: Boolean = false
7   )
```

*Tabel 27. Source Code LanguageModel.kt Modul 5*

```
1   package com.mobil.modul5compose.ui.screens.language
2
3   import kotlinx.serialization.Serializable
4
5   @Serializable
6   data class LanguageModel(
7       val name: String,
8       val tag: String
9   )
```

*Tabel 28. Source Code LanguageViewModel.kt Modul 5*

```
1a  package com.mobil.modul5compose.ui.screens.language
2
3   import androidx.appcompat.app.AppCompatDelegate
4   import androidx.core.os.LocaleListCompat
5   import androidx.lifecycle.ViewModel
6   import androidx.lifecycle.viewModelScope
7   import com.mobil.modul5compose.data.SettingsRepository
8   import timber.log.Timber
9   import kotlinx.coroutines.flow.MutableStateFlow
10  import kotlinx.coroutines.flow.StateFlow
11  import kotlinx.coroutines.flow.asStateFlow
```

```
12 import kotlinx.coroutines.flow.update
13 import kotlinx.coroutines.launch
14
15 class LanguageViewModel(
16     private val repository: SettingsRepository
17 ) : ViewModel() {
18                         private     val     _uiState     =
   MutableStateFlow(LanguageState(languages                  =
   repository.getAvailableLanguages()))
19              val   uiState:   StateFlow<LanguageState>   =
   _uiState.asStateFlow()
20
21     init {
22         observeLanguageChanges()
23     }
24
25     private fun observeLanguageChanges() {
26         viewModelScope.launch {
27             repository.languageTag.collect { tag ->
28                         Timber.d("LanguageViewModel: Observed
   language change in repository: %s", tag)
29                     _uiState.update { it.copy(selectedTag =
   tag) }
30             }
31         }
32     }
33
34     fun onLanguageSelected(tag: String) {
```

```
35            Timber.d("LanguageViewModel: User selected tag:
%s", tag)
36          AppCompatDelegate.setApplicationLocales(LocaleListC
ompat.forLanguageTags(tag))
37          repository.saveLanguageTag(tag)
38
39          _uiState.update { it.copy(navigateBackEvent = true)
}
40      }
41
42      fun onNavigateBackHandled() {
43              _uiState.update { it.copy(navigateBackEvent =
false) }
44      }
45  }
```

*Tabel 29. Source Code LanguageViewModelFactory.kt Modul 5*

```
1   package com.mobil.modul5compose.ui.screens.language
2
3   import androidx.lifecycle.ViewModel
4   import androidx.lifecycle.ViewModelProvider
5   import com.mobil.modul5compose.data.SettingsRepository
6
7   class LanguageViewModelFactory(
8       private val repository: SettingsRepository
9   ) : ViewModelProvider.Factory {
10      @Suppress("UNCHECKED_CAST")
11          override fun <T : ViewModel> create(modelClass:
```

```
    Class<T>): T {
12                                                          if
    (modelClass.isAssignableFrom(LanguageViewModel::class.java)
    ) {
13              return LanguageViewModel(repository) as T
14          }
15            throw IllegalArgumentException("Unknown ViewModel
    class")
16      }
17  }
```

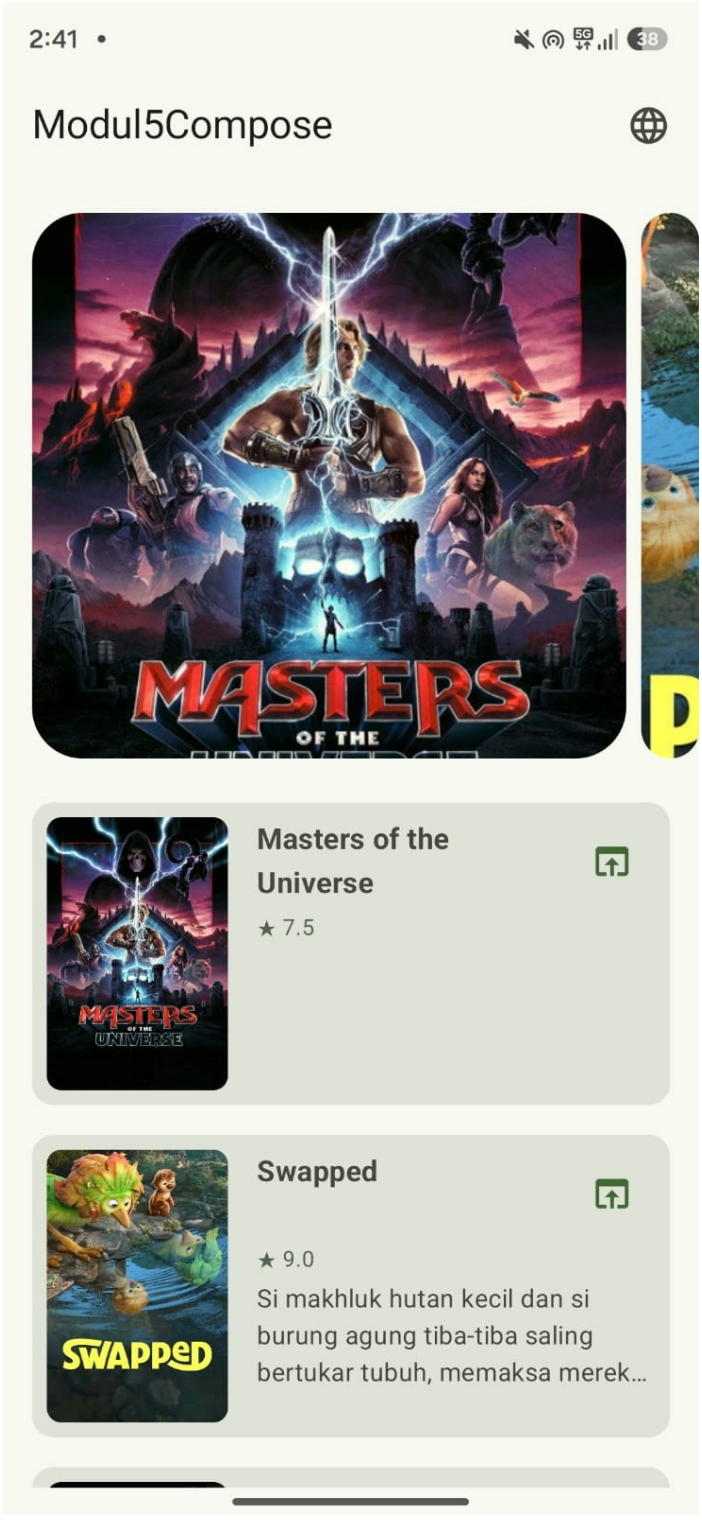
**9.Util**

*Tabel 30. Source Code MovieHelper.kt Modul 5*

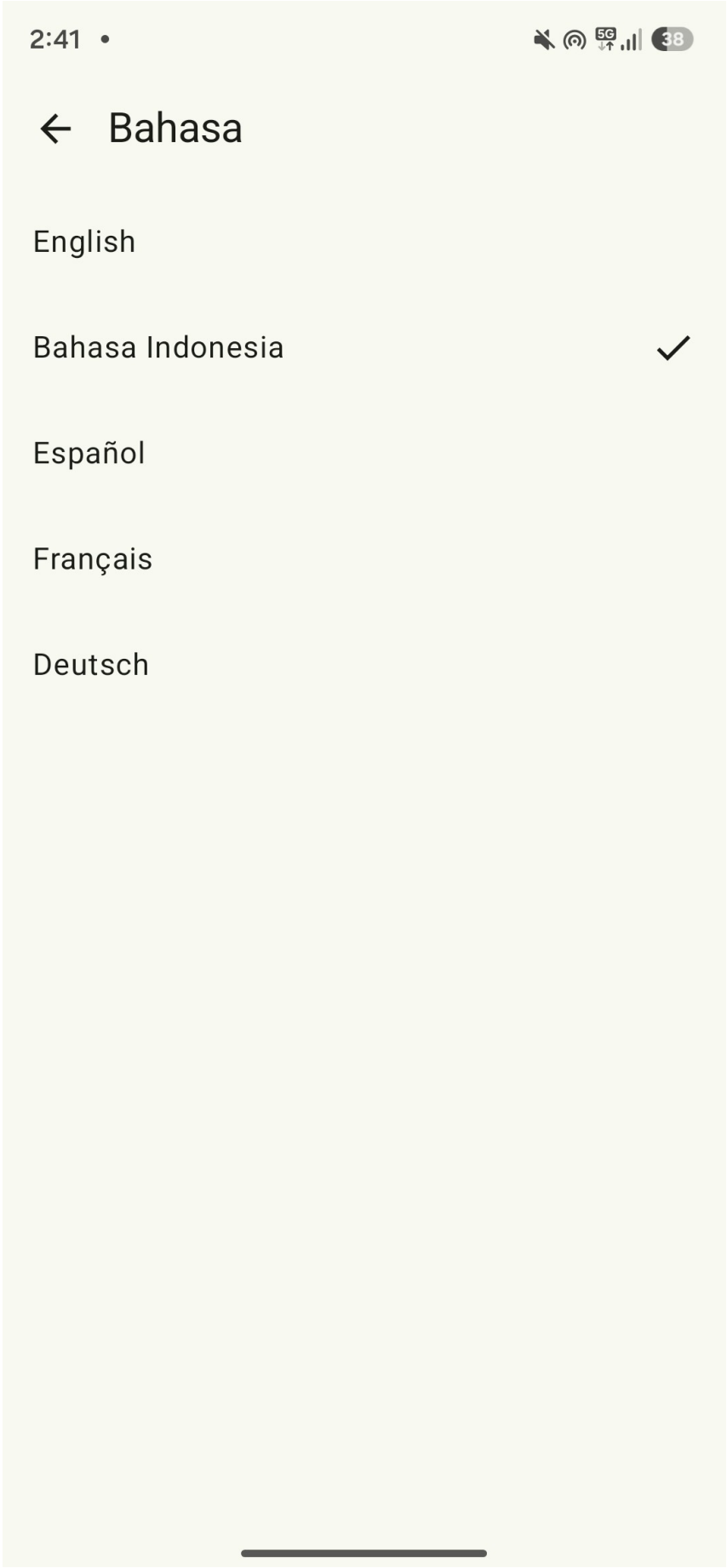
```
1   package com.mobil.modul5compose.util
2
3   import com.mobil.modul5compose.data.MovieResponse
4   import com.mobil.modul5compose.data.local.MovieEntity
5
6   fun       MovieResponse.toEntity(languageTag:       String):
    MovieEntity {
7       return MovieEntity(
8           id = this.id,
9               title  =  this.title  ?:  this.originalTitle  ?:
    "Unknown",
10          overview = this.overview ?: "",
11          posterPath = this.posterPath ?: "",
12          releaseDate = this.releaseDate ?: "",
13          voteAverage = this.voteAverage ?: 0.0,
```

```
14          languageTag = languageTag,
15          cachedAt = System.currentTimeMillis()
16      )
17  }
```

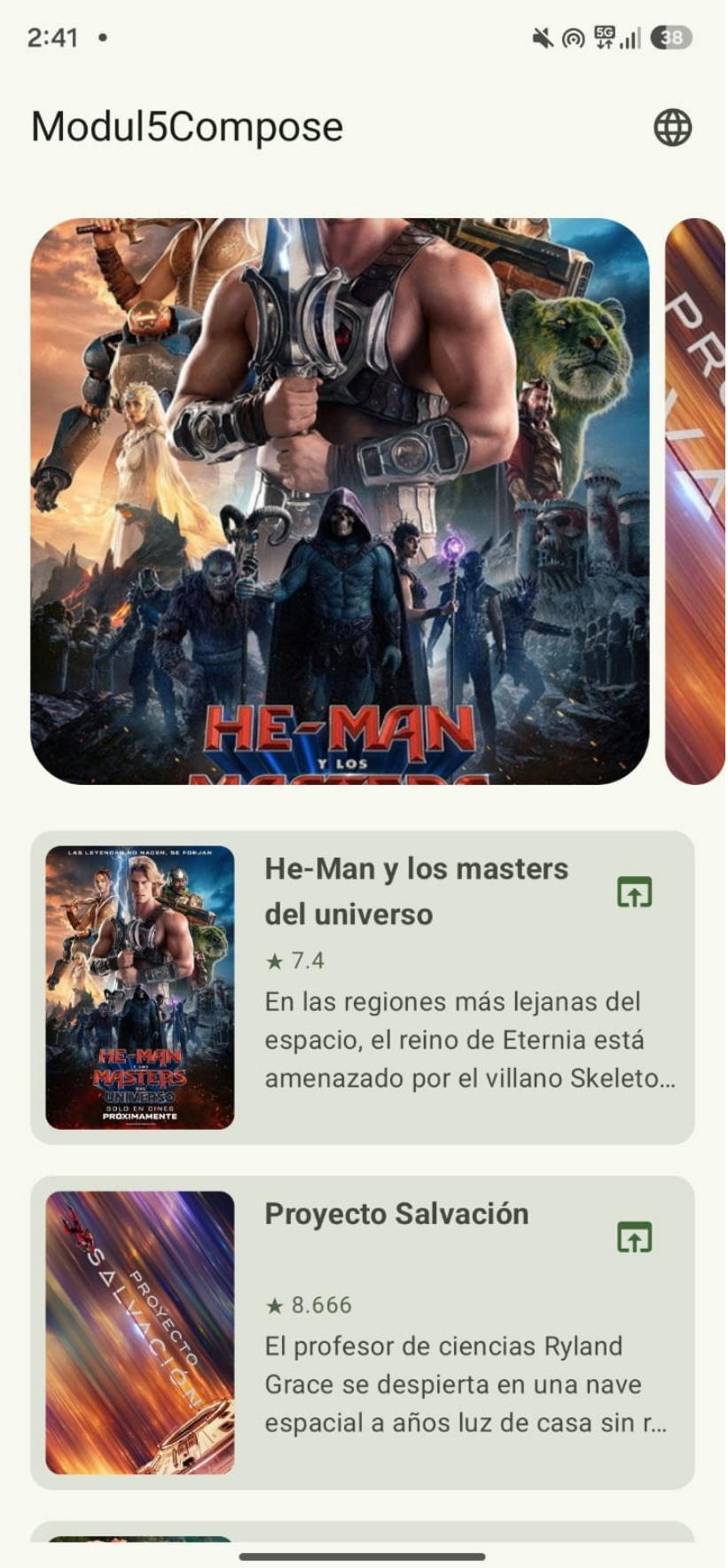
**B. Output Program**



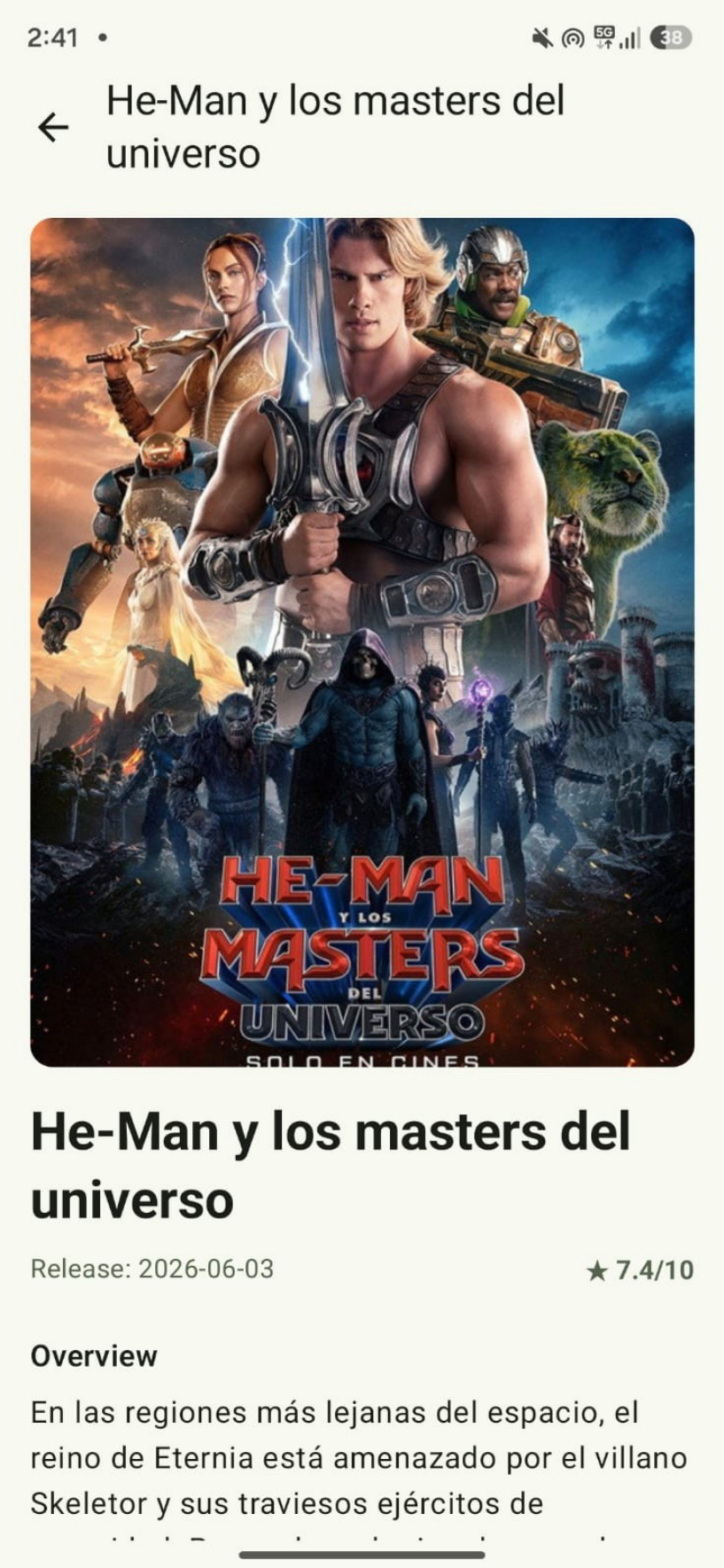

*Gambar 1. Output Screenshot Modul 5 (Indonesia)*



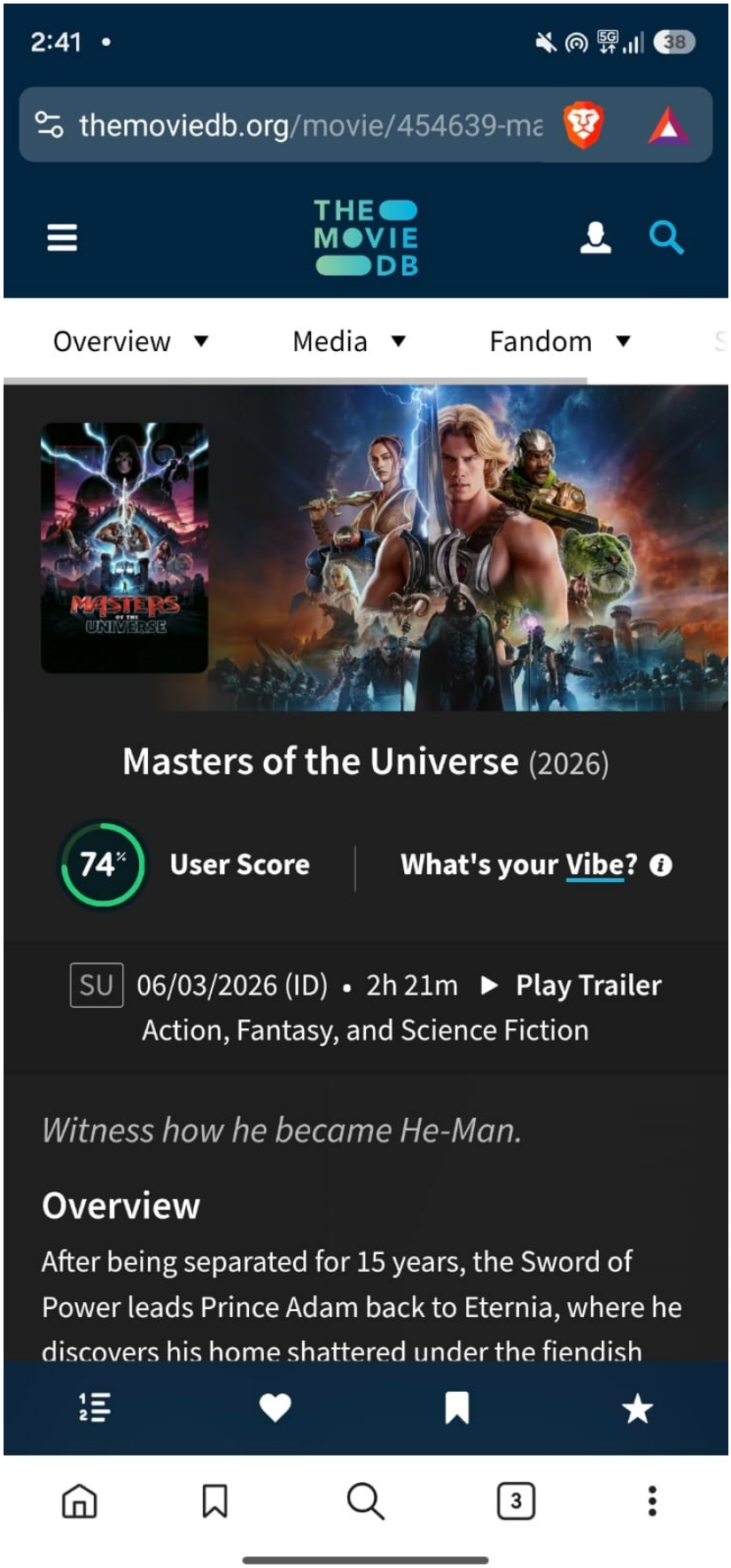

*Gambar 2. Output Screenshot Modul 5 (Settings)*



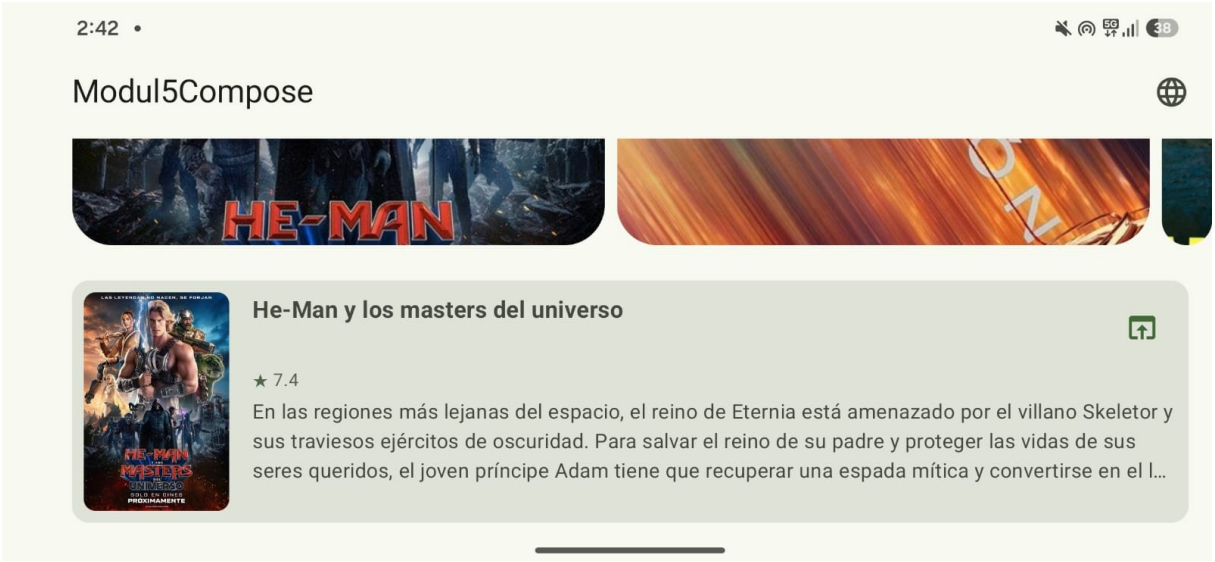

*Gambar 3. Output Screenshot Modul 5 (Spanyol)*



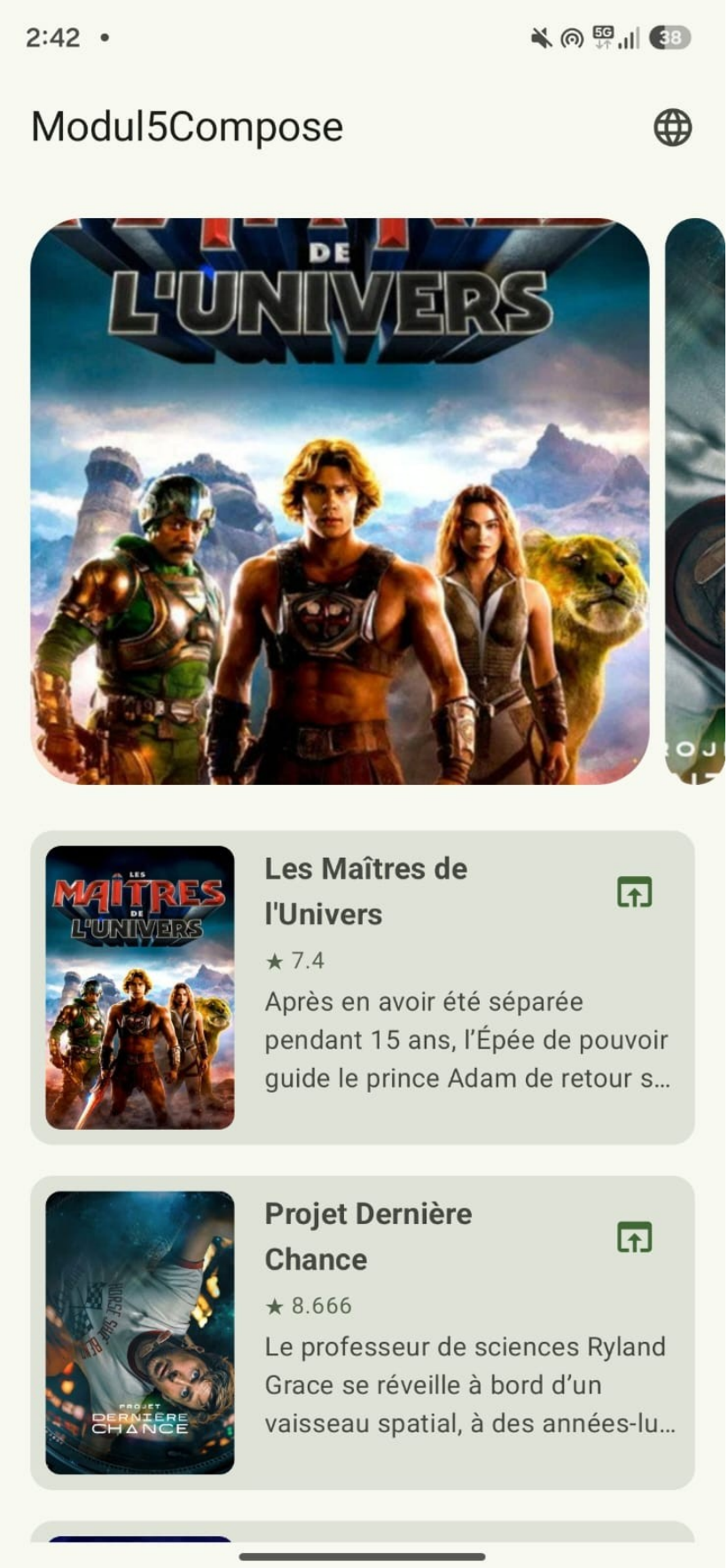

*Gambar 4. Output Screenshot Modul 5 (Detail Page)*



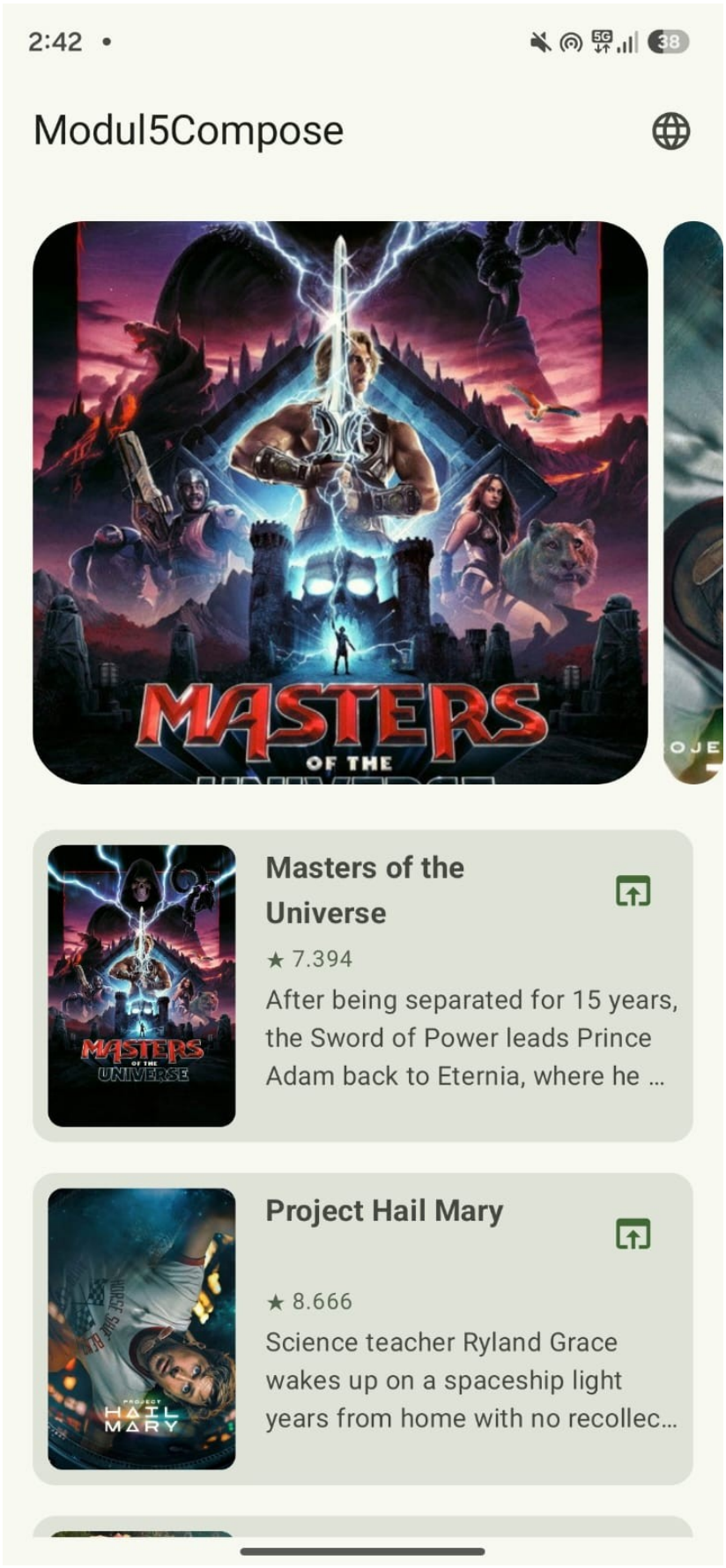

*Gambar 5. Output Screenshot Modul 5 (Implicit Intent)*



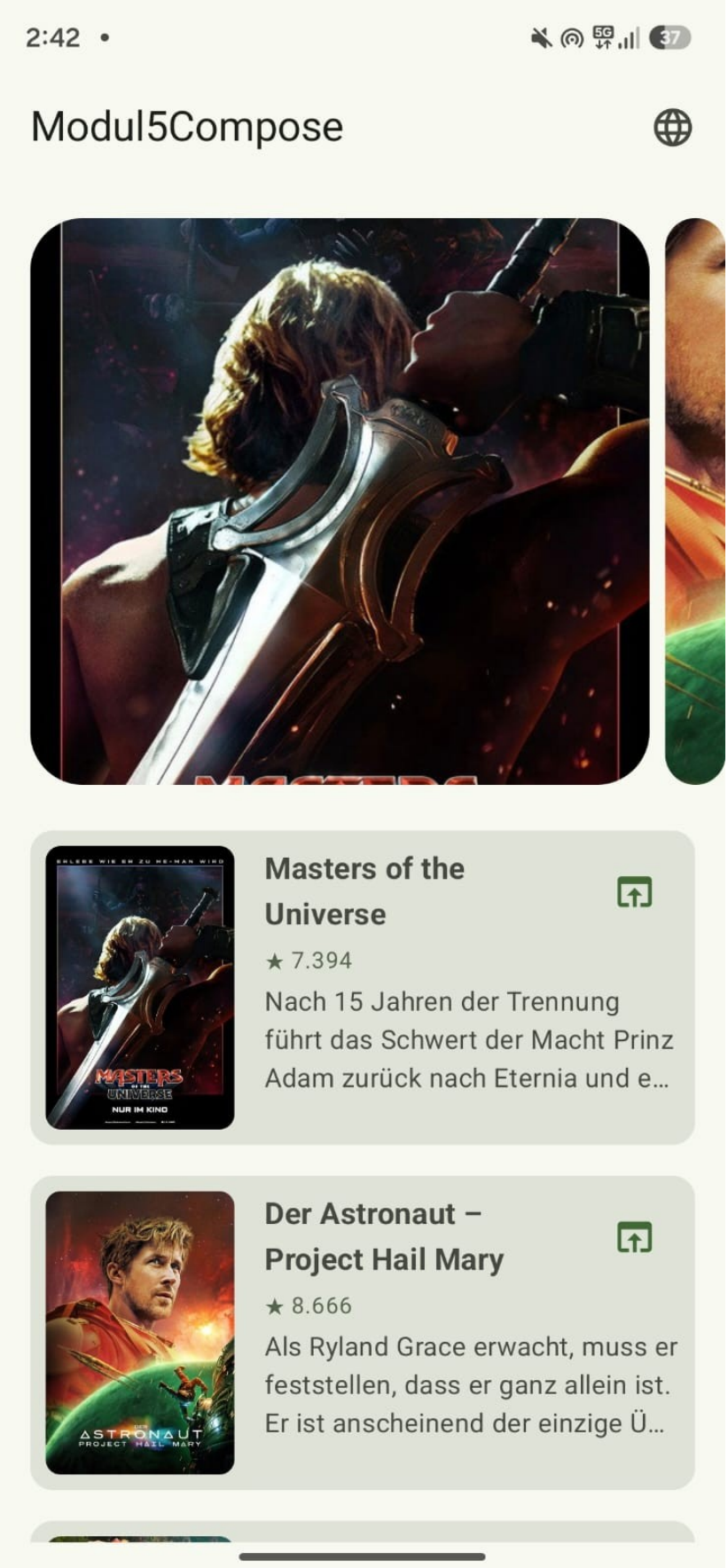

*Gambar 6. Output Screenshot Modul 5 (Landscape Orientation)*




*Gambar 7. Output Screenshot Modul 5 (Prancis)*




*Gambar 8. Output Screenshot Modul 5 (Inggris)*




*Gambar 9. Output Screenshot Modul 5 (Jerman)*

## C. Pembahasan

### Networking Library

Projek ini menggunakan Ktor sebagai networking library-nya. Ktor dipilih karena Kotlin-first, dan sangat fleksibel dalam konfigurasinya karena menggunakan DSL.

Client dibuat melalui implementasi HttpClientFactory yaitu TmdbClientFactory, sebenarnya bisa saja tanpa interface tapi saya ngebet mau mengaplikasikan prinsip Depedency Inversion dari SOLID. TmdbClientFactory menggunakan Engine CIO (Coroutines-based I/O) karena diberikan di contoh dokumentasinya. Base URL dan header Authorization disetting pada defaultRequest, jadi tiap kali Client melakukan request akan menggunakan base URL dan header Authorization yang sama.

Di MovieRepository, setiap fungsi memberikan data Flow<Resource<List<MovieEntity>>> atau Flow<Resource<MovieEntity>>, tidak langsung MovieEntity. Flow adalah Implementasi dari Oberserver Pattern untuk stream data secara Asynchronous, dan Resource adalah Model untuk komunikasi antara payload dan status operasi. Jadi, dengan menggunakan Flow<Resource<..>> UI bisa langsung berubah ketika terjadi perubahan di dalam MovieEntity tanpa perlu reload.

### KotlinX Serialization

Untuk urusan parsing JSON, projek ini menggunakan KotlinX Serialization yang diintegrasikan langsung dengan Ktor melalui plugin ContentNegotiation.

Model response dianotasi dengan @Serializable. Di dalam model, saya banyak menggunakan anotasi @SerialName. Tujuannya untuk memetakan nama field JSON bawaan TMDB API yang menggunakan snake_case ke naming convention Kotlin yang menggunakan camelCase. Sebenarnya bisa saja dikonfigurasi secara global untuk otomatis convert penamaannya, tapi saya lebih suka menuliskannya secara eksplisit seperti ini biar tidak ada magic yang tersembunyi dan mapping-nya jelas ketika dibaca.

Pada konfigurasi Json builder-nya, saya sengaja mengaktifkan ignoreUnknownKeys = true. Ini penting banget untuk robustness. Efeknya, kalau sewaktu-waktu TMDB mengubah API mereka dan tiba-tiba mengirimkan field tambahan yang belum kita definisikan di model, aplikasi kita tidak akan langsung crash gegara gagal deserialization.

Selain itu, saya juga mendefinisikan default value berupa null pada banyak field (misal val title: String? = null). Pendekatan ini menjaga sistem tetap aman kalau ada data tertentu yang tidak dikirimkan dalam respons API. Intinya, kode ini dirancang agar long-lasting dan tidak gampang meledak dan hancur di production hanya karena masalah dari backend.

Anotasi @Serializable ini tidak cuma saya manfaatkan untuk urusan networking. Saya juga menggunakannya di RoutingNames.kt dan LanguageModel.kt. Tujuannya untuk mengimplementasikan type-safe navigation menggunakan Navigation Compose. Jadi, ketimbang pusing dan rentan typo karena melempar argumen navigasi pakai tipe String biasa, serialisasi data yang ringan ini membuat proses passing data antar view jadi jauh lebih clean dan aman dari error saat compile time.

**Image Loading Library**

Untuk urusan memuat gambar poster film dari TMDB di HomeScreen.kt dan DetailScreen.kt, saya memutuskan untuk menggunakan Coil (Coroutine Image Loader). Sebenarnya bisa saja pakai library legendaris seperti Glide atau Picasso, tapi karena saya fokus membangun arsitektur yang long-lasting dan sejalan dengan ekosistem modern, Coil adalah pilihan yang jauh lebih masuk akal. Coil ini dirancang Kotlin-first dan berjalan murni di atas Coroutines, jadi benar-benar terasa native dan solid ketika dikawinkan dengan arsitektur Jetpack Compose.

Di sini saya menggunakan composable AsyncImage bawaan dari coil-compose. Eksekusinya memuat gambar secara asinkron murni tanpa memblokir UI thread sama sekali.

Dari sisi robustness, menggunakan Coil menyelamatkan saya dari banyak kerumitan sistem. Library ini secara otomatis menangani layered cache dan menampilkan placeholder saat gambar sedang dimuat. Namun, dampak terbesarnya bagi performa aplikasi secara keseluruhan adalah kemampuannya membatalkan request secara otomatis ketika composable keluar dari komposisi layar misalnya saat user melakukan scroll dengan cepat. Ini krusial banget untuk mencegah memory leak dan memastikan aplikasi tidak rakus resource.

Untuk URL path-nya, saya merangkainya langsung dari TMDB dengan mematok ukuran w500. Ini adalah keputusan pragmatis; ukurannya sudah sangat pas dan cukup tajam untuk merender tampilan list maupun detail. Jadi, kita tidak perlu membuang-buang kuota user atau mengorbankan performa rendering dengan mengunduh file gambar yang kelewat besar.

**TMDB API**

Projek ini mengonsumsi The Movie Database (TMDB) API dengan dua endpoint utama.

Endpoint yang digunakan:
client.get("movie/popular?language=$currentLanguage")
client.get("movie/$movieId?language=$currentLanguage")

Parameter language dikirim secara dinamis sesuai preferensi user yang tersimpan di SettingsRepository. Autentikasi menggunakan Bearer Token (Read Access Token) yang disisipkan di header setiap request lewat defaultRequest di TmdbHttpClientFactory. Token disimpan aman di secrets.properties, tidak di-commit ke repositori, ada file secrets.properties.example sebagai panduan.

Response dari endpoint list disimpan dalam TmdbListResponse yang memiliki atribut result: List<MovieResponse>, kemudian di-mapping ke MovieEntity melalui extension function toEntity() di MovieHelper.kt sebelum disimpan ke Room.

**Caching Strategy**

Projek ini menggunakan strategi Cache-First with Network Fallback yang diimplementasikan melalui pola NetworkBoundResource.

Alur kerjanya, pertama membaca cache dari Room, lalu memutuskan apakah perlu fetch ke network berdasarkan shouldFetch. Untuk daftar film, fetch dilakukan hanya jika cache kosong (cachedList.isNullOrEmpty()). Untuk detail film, fetch dilakukan jika data belum ada di cache (cachedEntity == null). Jika fetch berhasil, data disimpan ke Room dan langsung di-emit. Jika gagal, cache lama tetap ditampilkan bersama pesan error.

Alasan saya memilih strategi ini karena data film dari TMDB tidak sering berubah, daftar film populer cukup stabil dalam jangka pendek. Dengan strategi ini, aplikasi tetap dapat digunakan saat offline dengan menggunakan data cache, dan UI tidak perlu menunggu network sebelum menampilkan konten. Selain itu, MovieEntity menyimpan field cachedAt: Long yang memungkinkan development lebih lanjut untuk invalidasi cache berdasarkan time-based expiry.

**UI**

Sistem UI projek ini saya bangun full menggunakan Jetpack Compose. Saya sengaja menerapkan arsitektur MVVM dengan batasan yang sangat tegas antara Screen, ViewModel, dan State. Kenapa? Supaya kodenya long-lasting, scalable, dan gampang di-test. Komponen layarnya saya pecah jadi tiga: State, ViewModel, dan Screen.

Untuk urusan navigasinya, sebenarnya bisa saja pakai string route biasa yang lebih cepat dibikin, tapi saya ngebet mau mengaplikasikan type-safe routes bawaan Navigation Compose terbaru. Dengan mendefinisikan route sebagai sealed class beranotasi @Serializable di RoutingNames.kt, saya bisa memastikan aplikasi aman dari crash gak seru hanya gara-gara typo nama screen atau argumen yang salah tipe data.

**HomeScreen**

Di HomeScreen, state dari ViewModel saya konsumsi menggunakan collectAsState(), jadi UI akan langsung recompose secara otomatis tiap kali datanya berubah.

Tantangan di Compose biasanya ada di navigation event. Kalau tidak hati-hati, layar bisa pindah berkali-kali saat terjadi recomposition. Untuk mengakalinya, event navigasi ini saya modelkan sebagai field nullable di dalam HomeState. Ketika nilainya tidak null, navigasi langsung diproses di dalam LaunchedEffect, lalu di-reset kembali ke null.

Secara tampilan, saya membaginya jadi dua: MovieCarousel di atas yang menggunakan HorizontalMultiBrowseCarousel sebenarnya ini API Material3 yang masih experimental, tapi saya pakai karena cocok untuk menampilkan 5 film, dan LazyColumn di bawahnya untuk daftar film biasa.

Di level arsitektur yang lebih luas, HomeViewModel terus-terusan mengobservasi perubahan languageTag dari repository. Efeknya sangat smooth, setiap kali user mengganti bahasa, job pengambilan data yang lama akan dibatalkan, dan data baru dengan bahasa yang sesuai akan langsung di-reload secara asinkron tanpa mengganggu flow UI.

**DetailScreen**

Masuk ke DetailScreen, layar ini bertugas menampilkan informasi lengkap film berdasarkan movieId yang dipassing secara type-safe tadi.

Begitu DetailViewModel menerima movieId, dia akan langsung memanggil repository.getMovie(movieId). Nah, di repository inilah keajaibannya terjadi. Saya menerapkan pola NetworkBoundResource. Pola ini membuat aplikasi jadi kebal terhadap koneksi internet yang jelek. Sistem akan langsung menampilkan data cache dari database lokal sambil melakukan fetching API di background.

Kalau API tiba-tiba error atau internet mati, aplikasi tidak akan crash atau menampilkan blank screen. Film tetap ditampilkan dari cache, dan user hanya diberikan pesan error kecil di overlay. Ini benar-benar menjaga robustness aplikasi di production.

**LanguageScreen**

LanguageScreen bertugas mengurus preferensi bahasa yang persistent. UI-nya cukup simpel dengan LazyColumn berisi opsi bahasa.

Ketika user memilih bahasa, LanguageViewModel mengeksekusi dua tindakan krusial. Pertama, dia memanggil AppCompatDelegate.setApplicationLocales() untuk mengubah locale aplikasi di level sistem, sehingga semua string resources bawaan langsung diterjemahkan. Kedua, memanggil repository.saveLanguageTag(tag) untuk menyimpannya ke SharedPreferences supaya preferensinya tidak hilang saat aplikasi mati.

Karena SettingsRepository mengekspos bahasa ini dalam bentuk StateFlow<String>, sistem menjadi murni reaktif. Semua Screen dan ViewModel yang bergantung pada bahasa seperti HomeViewModel, akan langsung bereaksi terhadap perubahan tanpa perlu kita perintahkan. Hasil akhirnya, UI aplikasi akan berubah bahasa on-the-fly secara elegan tanpa perlu merestart aplikasi.

# LINK GITHUB

https://github.com/Gunturadhtya/Praktikum-Pemrograman-Mobile